

FIT2098 - Documentation

By Yash Saksena - 33680442

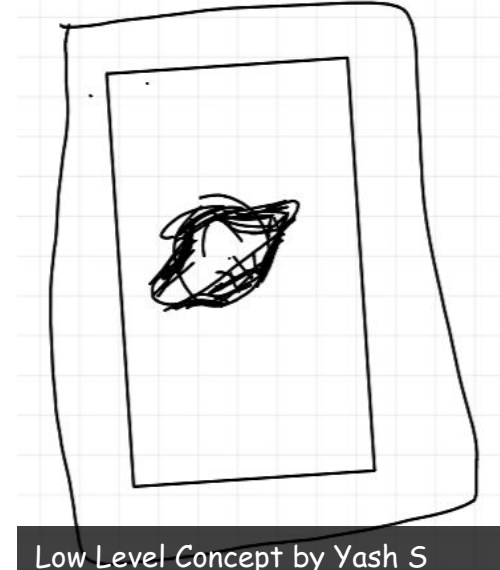
Making a world hidden in the real world...

I was tasked with the amazing prospect of creating a universe inside of another (in MR) , and my first couple thoughts were leaning towards a sci-fi environment, as they have always been interesting to me.



First Concept.

The first concept was in some ways similar to Half-life 1 (p2) where the main character triggers a cataclysmic effect by opening a door/portal into a universe. I was thinking that the environment will have a black hole, and eventually it takes over the experience of the user, but I feel as though this would be too complicated to complete with the number of graphical effects needed. So I moved on to another concept.



Second Concept

While thinking about what to do I saw my portable hard drive sitting on my desk, and I had the idea of making a circuit world through the drive.

I initially had the idea of creating a sort mind of a computer as a world. So it would have like the memories of screenshots or random videos. And it would have all the different random files you would find in a computer. But I realised that that might be a bit too abstract of a concept to create.

After consoling with Min I decided to continue in this direction, and remembered an episode on "The Orville", where people found a 2 dimensional species. I thought what if I created a world where technology was sentient, in the literal sense. So like the components of a motherboard running around and doing things or reacting to the user.



The Orville (Show) by Seth MacFarlane

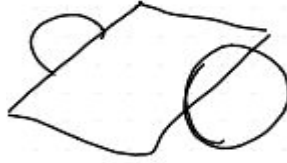


The Drive by Yash S

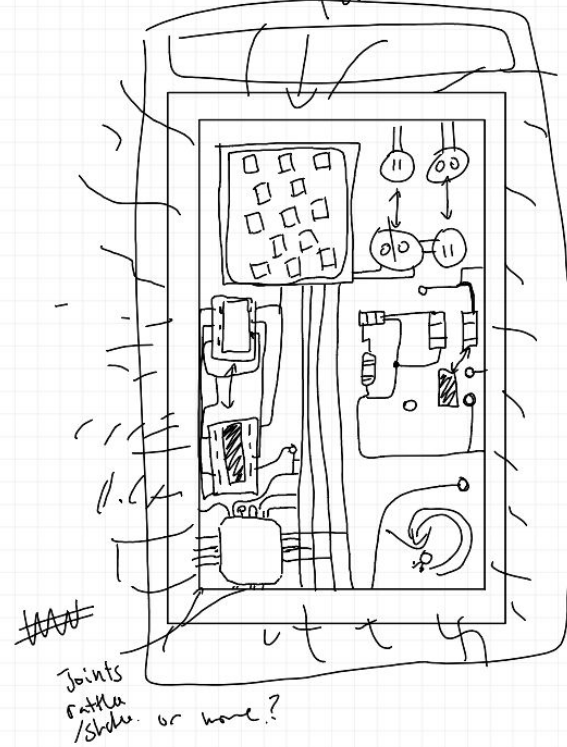
Concept Art

Main Ideas I came up with

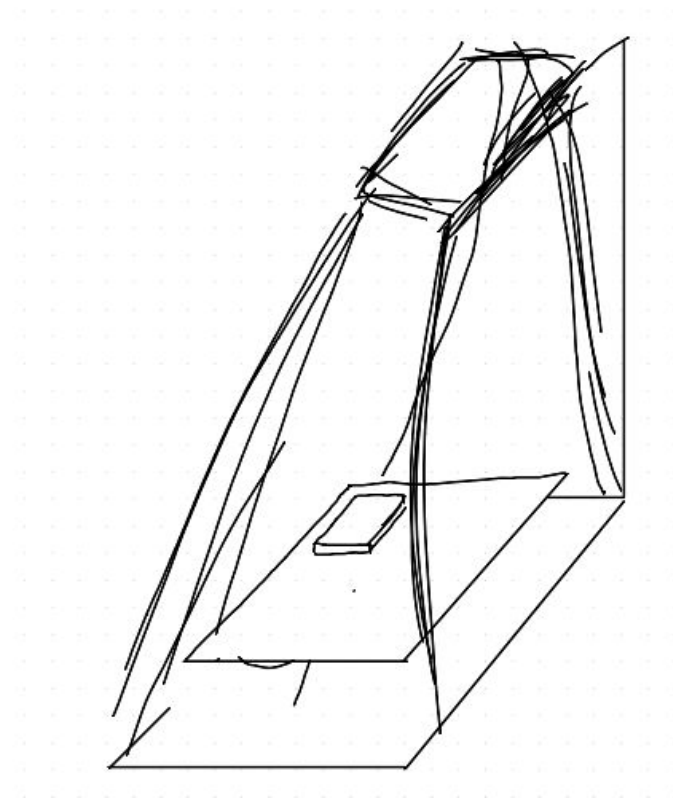
1. Moving components
 - a. Jumping between sockets
 - b. Going around the map.
2. Glowing lights
3. Screen with moving led's
4. Matrix Rain ([#48ff1e](#)) Effect for the border



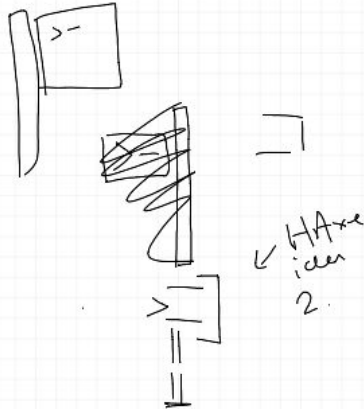
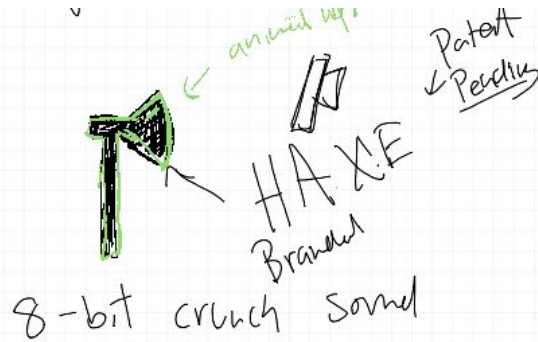
Hacking (literally)
Door = Face? (animated)



More Concept Art



The Key



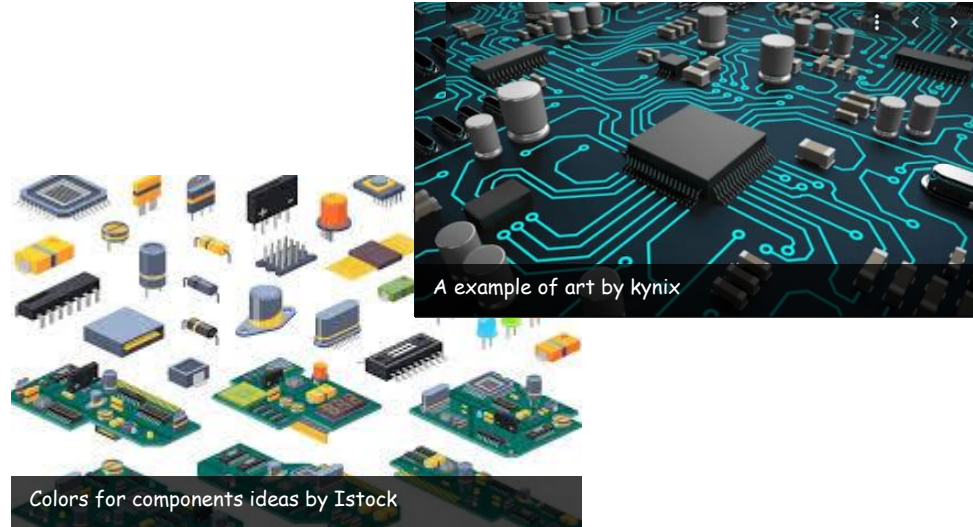
Thematically I thought the idea of hacking would be cool. Then I thought of hacking as a verb, and realised I could likely make a axe that you use to hack into the drive. ****literally****

AND I'M CALLING IT THE HAXE

Colours and sfx

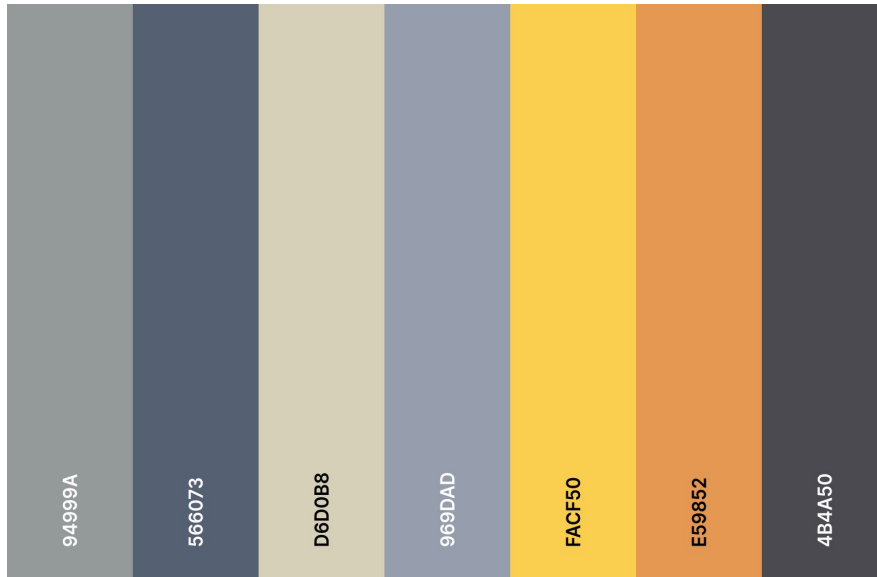
I think using colours like cyan and green, with a matrix rain effect in the background will create a very engaging environment.

In terms of art style I was initially thinking of low poly similar to this isometric image, however, I though aesthetically speaking having a voxel based artstyle would be more appropriate. And this would also include the sfx being 8-bit or retro.



Colour Pallets (Derived from images)

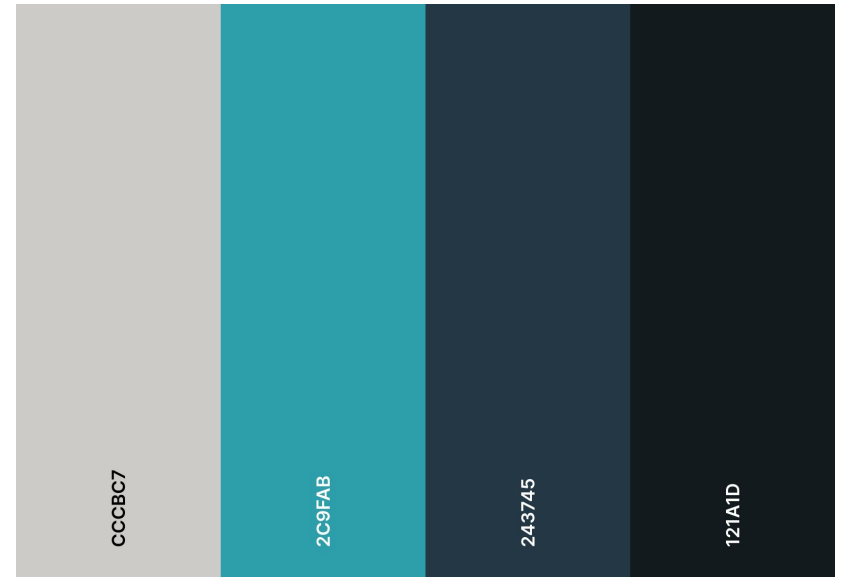
Component Colours



Component Colours

coolors

PCB Colours



PCB Colours

coolors

Measurements

Drive Case:

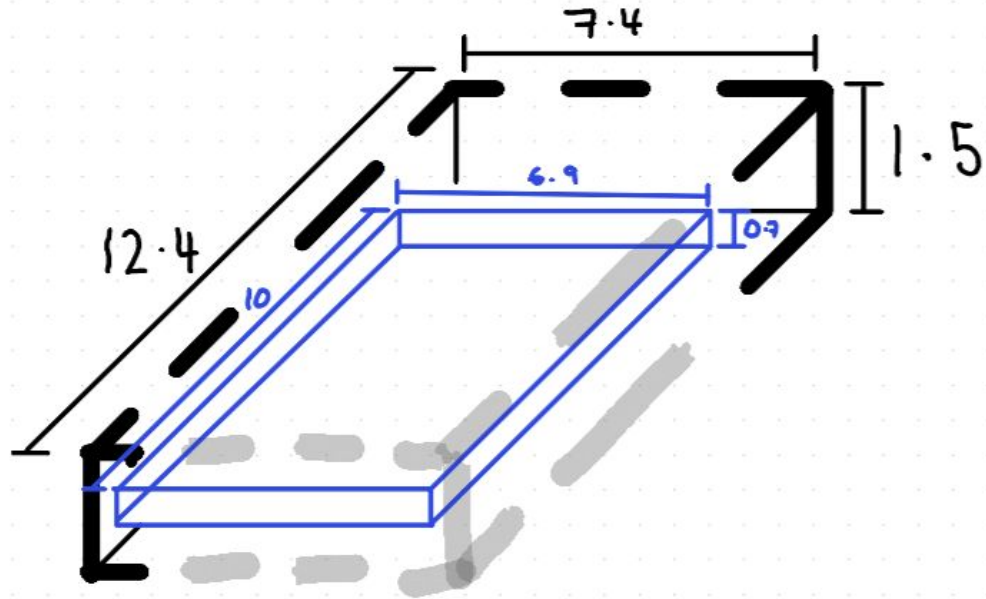
- ★ Depth = 12.4cm
- ★ Width = 7.4cm
- ★ Height = 1.5cm

Drive:

- ★ Depth = 10cm
- ★ Width = 6.9cm
- ★ Height = 0.7cm

Diagram *not* to scale

All measurements in CM.

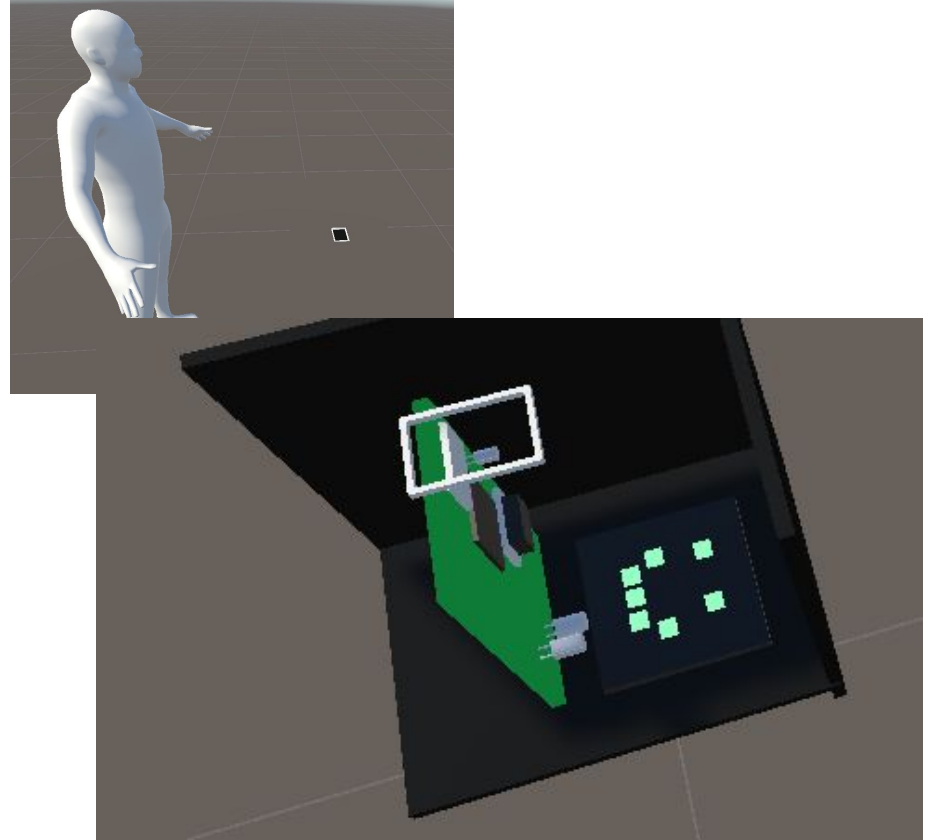


Block out creation

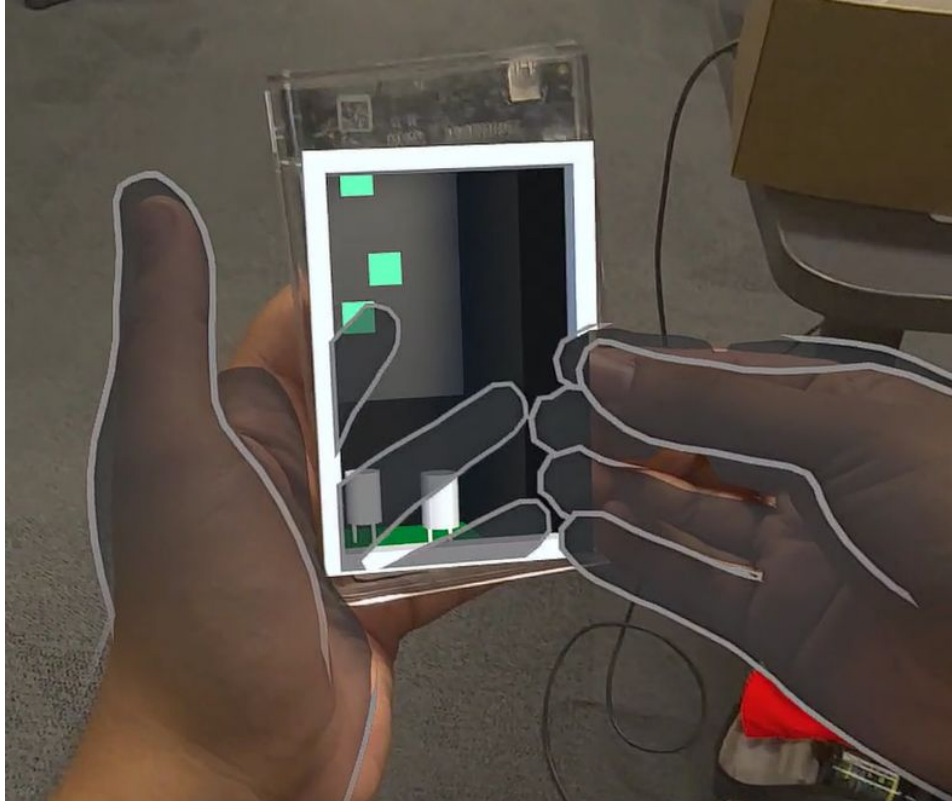
During the block out process I created what I drew and the next day I tested it in the lab. However, while doing so I realized that maybe the perspective I gave the world was not good enough so I changed it up a little bit.

So that the player will be looking into the plane rather than looking above it.

My primary reasoning for this is so that the player can see the world looking back. It also gives a closer perspective of what's happening around the world.



Scale Test

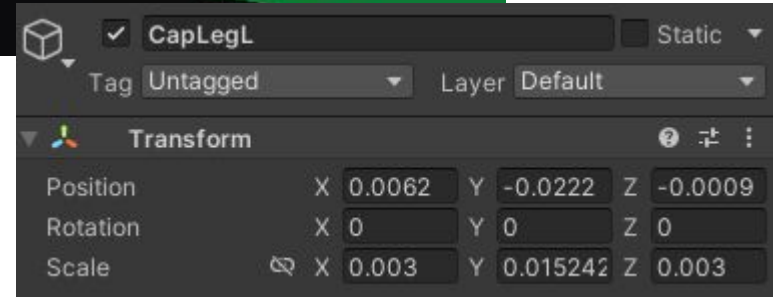
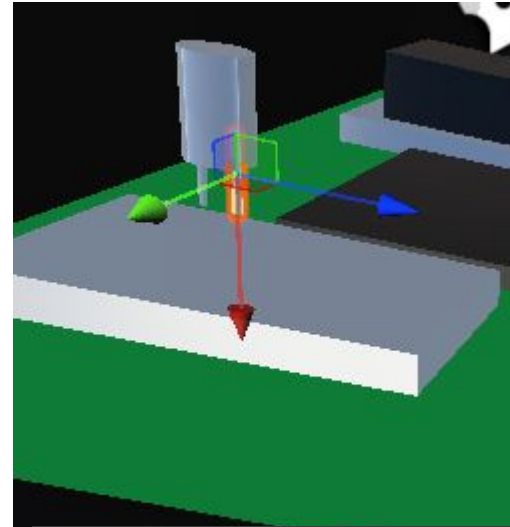


<https://youtu.be/l6hHu2gpiFE>

Determining The Voxel Size

As you would likely know the word voxel comes from Volume Cell in the same way the word Pixel comes from Picture Cell. So inherently 1 voxel would have a fixed height, width and length.

In the case of this I decided to set that distance to 0.3cm as that was the diameter of the capacitor legs that I made in my block out.



Figuring out the workflow.

While approaching creating voxel based art, I read that I had to use MASH (which is a node based system for modelling in Maya). However, the only real info i could get on using mash was a video with Maya 2007, which was completely irrelevant in the context of Maya 2025.

When doing more research, I found a video of someone, converting a 3D model of OBJ format into a Voxel Model with a program called MagicaVoxel, which automatically converted the model into a Voxel version.

This seemed like the only approach feasible at the time of creation. And MagicaVoxel is generously free to use for any project. So I think it should be fine for this assignment.

An added perk to this workflow is that the models in magicavoxel were exported with textures, this was a great help as I could use MagicaVoxel to both texture and finalise the models.

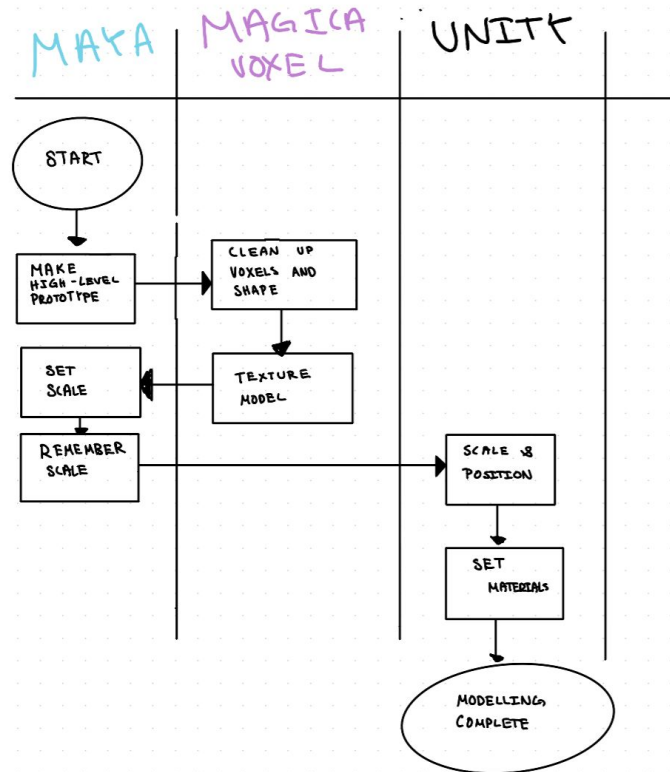
Not So Precise Voxels...

As I began working on the models, i realised that when importing them into unity the models would lose the scale I gave them in Maya.

So I decided that I would texture the models in MagicaVoxel and then proceed to find the correct rescale of the model by importing the model back into Maya, and after that importing the model to unity and rescale the model accordingly.

This also means that I can't exactly have a fixed Voxel size as MagicaVoxel doesn't seem support such a feature.

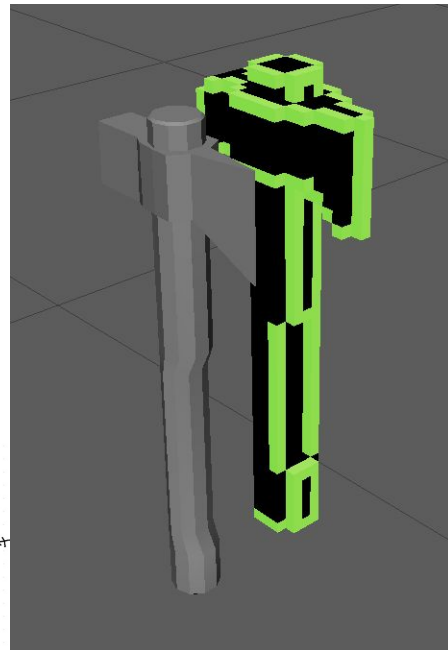
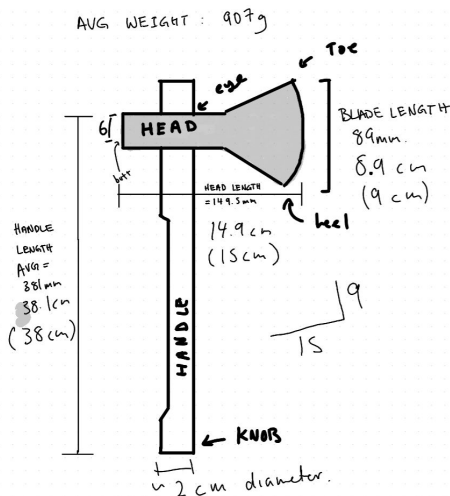
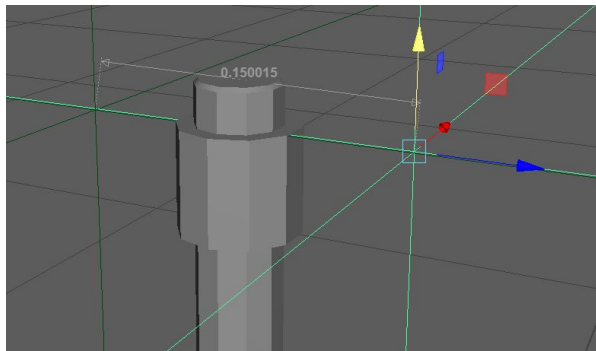
Final Modelling Workflow:



Creating The HAXE

As outlined in the workflow, I first created the axe in maya then imported it into MagicaVoxel after which I textured it and made sure that the silhouette of the model was to my liking.

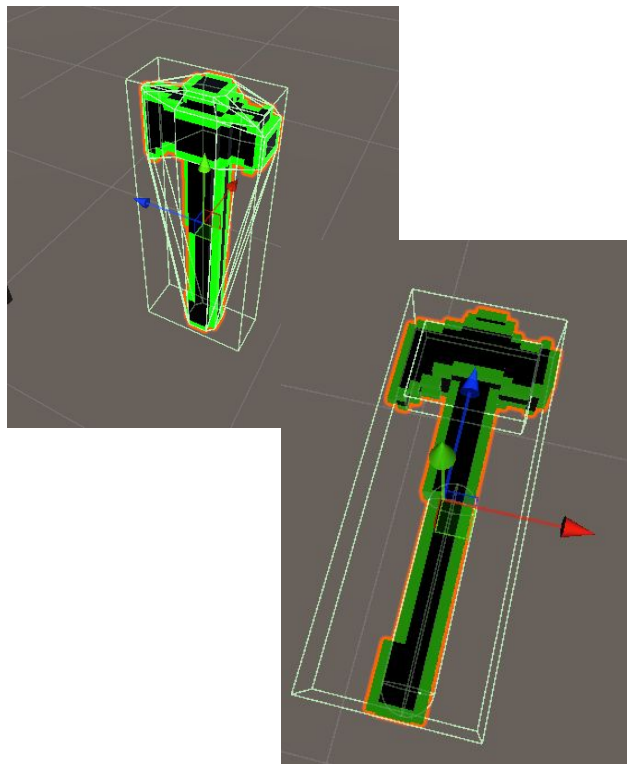
I also used measurements I found from the [International Axe throwing Federation.](#)



Applying key and trigger attributes to the Axe

Initially I decided to use the Complex mesh Collider to complete the interactable hand grab script. However, I realised this made the hand grab highlight really weird, and inaccurate which broke the immersion. So I decided to instead use a simple Capsule Collider instead and aligned it with the handle of the axe.

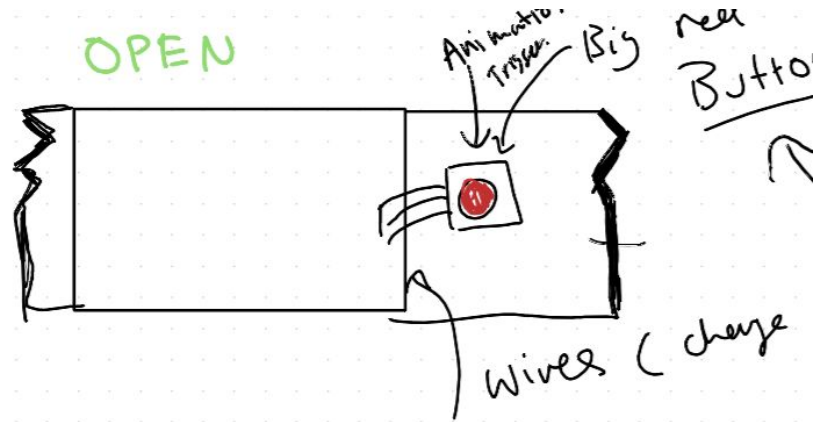
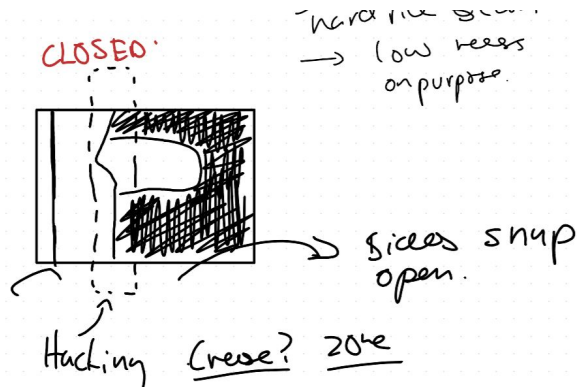
I also further added a second box collider to act as the trigger, so that the door opening animation will only trigger when it hits the actual blade section of the axe.



Door Ideation.

As mentioned the door itself here will be the portable hard drive, and my idea was basically to split the metal and board parts between the drive to show the world.

Ignore the red button, that was a idea that I didn't follow through with



World - Creating the door.

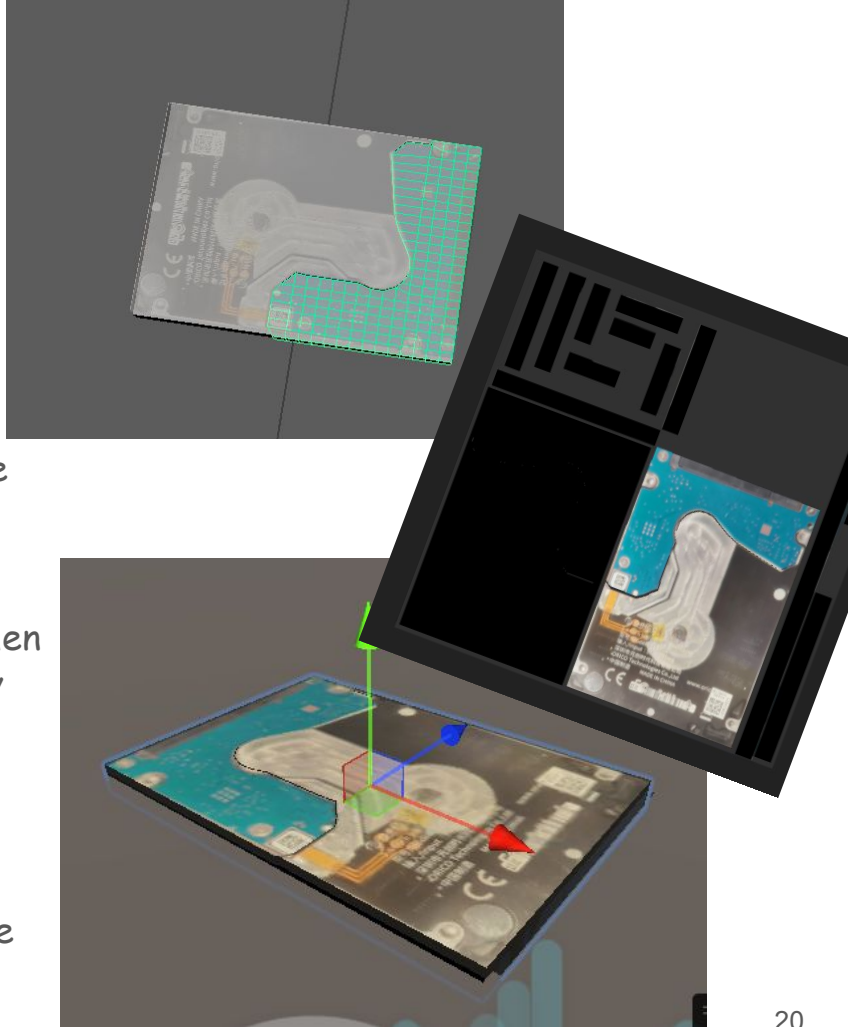
I created a plane the size of the drive, and gave it 25x25 partitions. This was so that I can make an accurate split between the board section and the metal section of the drive.

I then added an image plane onto the plane and traced the outline of the board section. (by moving the vertexes)

After that I selected all the faces for the board sections and created a new object by cutting and pasting it. This then gave me two planes that I was able to extrude into exactly what I needed to make the door work.

I then grouped them and aligned them, UV Mapped them, and then exported into fbx.

Whereafter I used substance painter to drag a scan of the drive into the texture and align it with the objects.



A different design approach - Kitbashing

In props and set creation for films, something people do is kitbashing, this is when they take different parts from different models and kits and combine them to create a unique model.

In a similar way I wanted to make a component based system that I could use to create my environment with very similar parts and pieces.

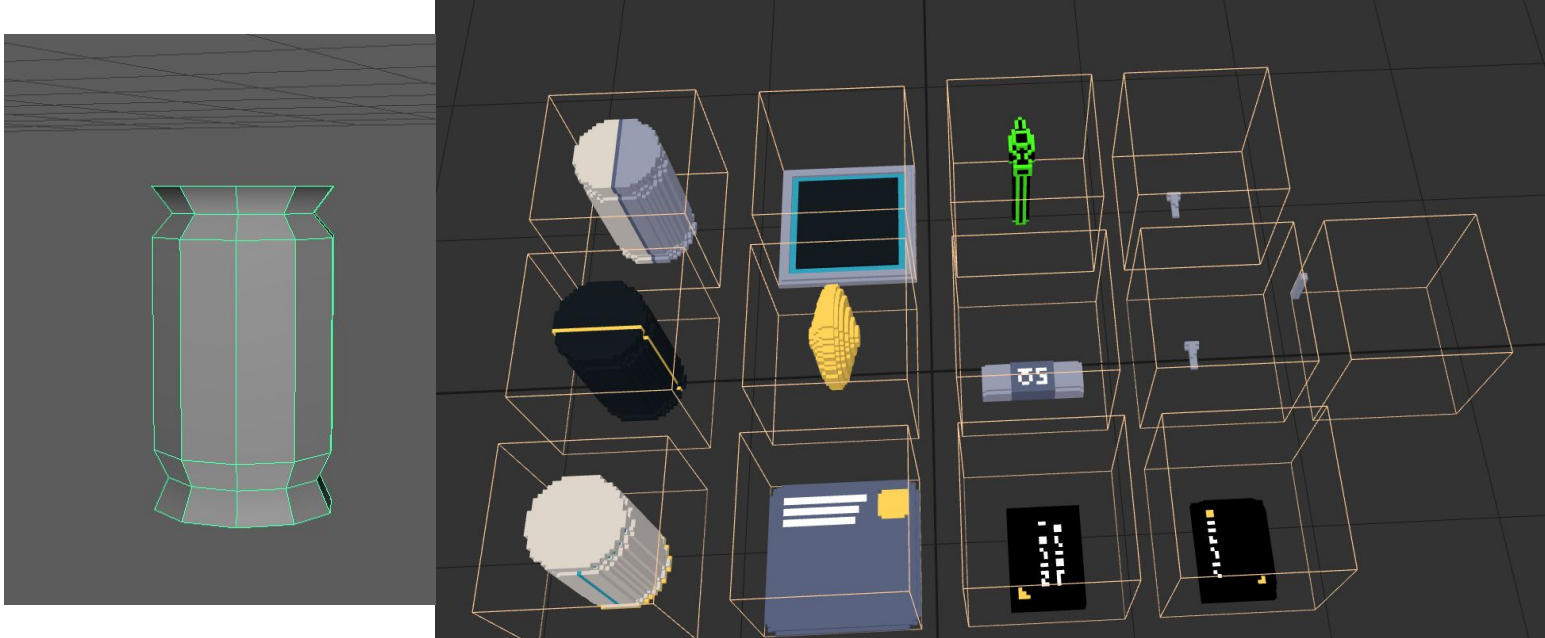
An advantage of doing this is also that I don't have to necessarily worry about the scale of the objects. As I can scale them in engine. Furthermore, I would have to create fewer models, and I can focus more on the design of the environment.



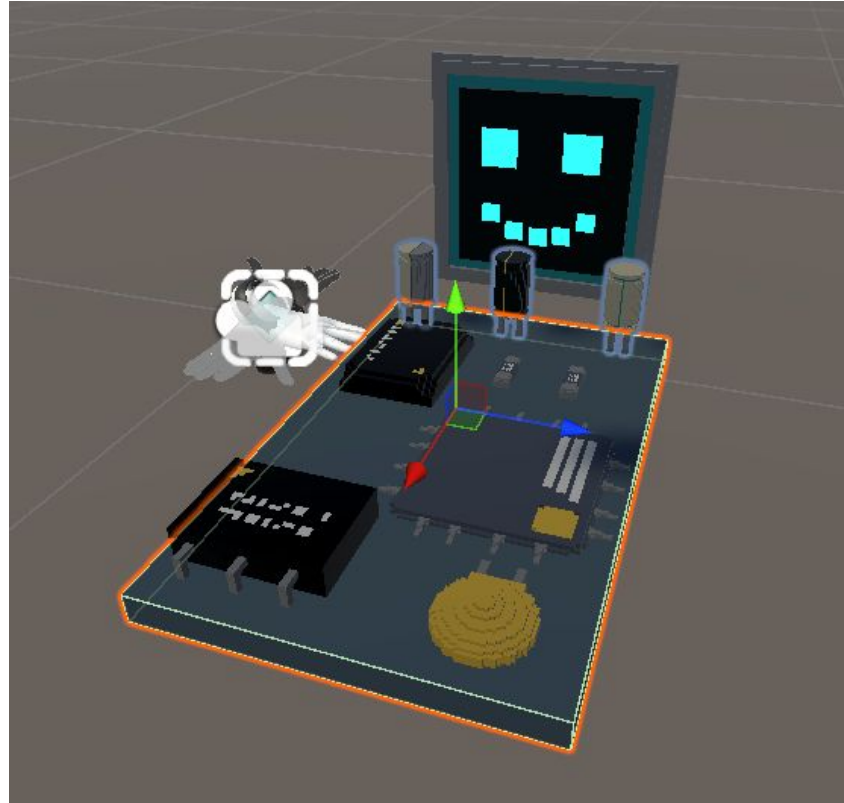
Adam Savage Kitbashing IRL in [his video](#)

World - Designing the "Kit".

I decided to use MagicaVoxel directly for some of the models as they were quite simple shapes.



World - Throwing it all together.

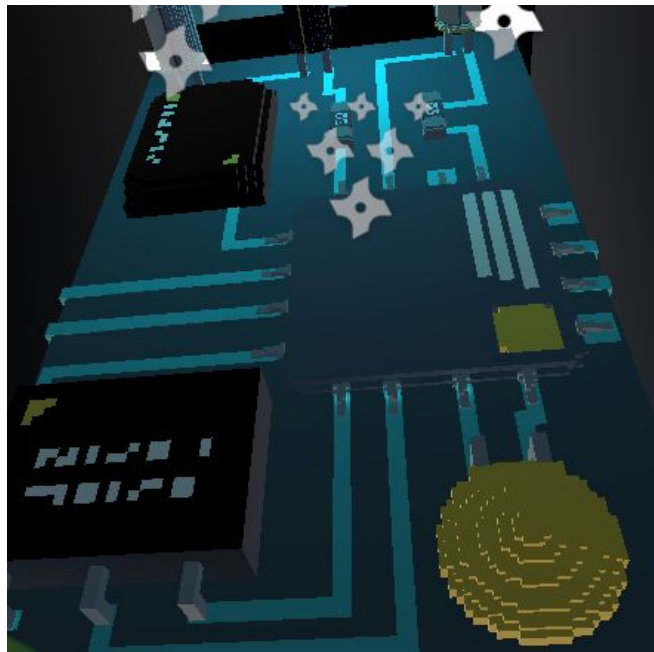


World - Cheating with planes and cubes.

I just made a shader material that used the “unlit” colour of the wire from the reference images, and I also used a lit version of the texture by adding emission and giving the HDR color a +2 with intensity

I then used Planes and cubes to represent wires and electrons. These would then be animated to move around the world.

I decided to animate these as they would help bring more life into the world, and also acting as light sources they would make the environment more dynamic.

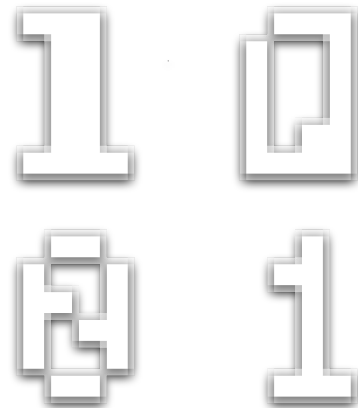


Particles - Matrix Texture

As mentioned I wanted the Matrix Rain Effect to be on the exterior of the world. So I made it.

I started with a 16x16 pixel image in photopea where I made simple pixel art for two types of 1's and two types of 0's. I then went into Paint.Net and resized the image with nearest neighbour to 512x512, so that the pixel art would stay as it is. Then I used a fractalize filter to give the art a semi opaque border.

This texture was then imported into unity for use with the particle system.



Particles - Matrix Particle

It was quite simple to make the particle system.

I first created a shader for the particle, and used the texture I created earlier.

I then imported the texture into the render section for the particle system.

Things I enabled:

- Texture sheet animation,

Modifications:

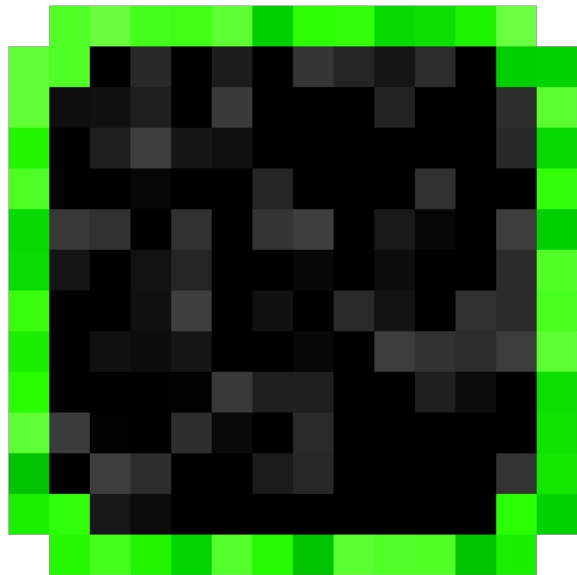
- I reduced the lifetime, size and shape such that the texture would work seamlessly with the border of the world.
- I also made the texture follow the face of the shape, making it face the front rather than the camera.



Particles - Dust Texture;

Much the same as the matrix texture I simply created a smaller scale texture and upscaled it.

This time instead of using the fractalize filter I used a noise filter to make the particle look more dithered.



Particles - Dust Particle

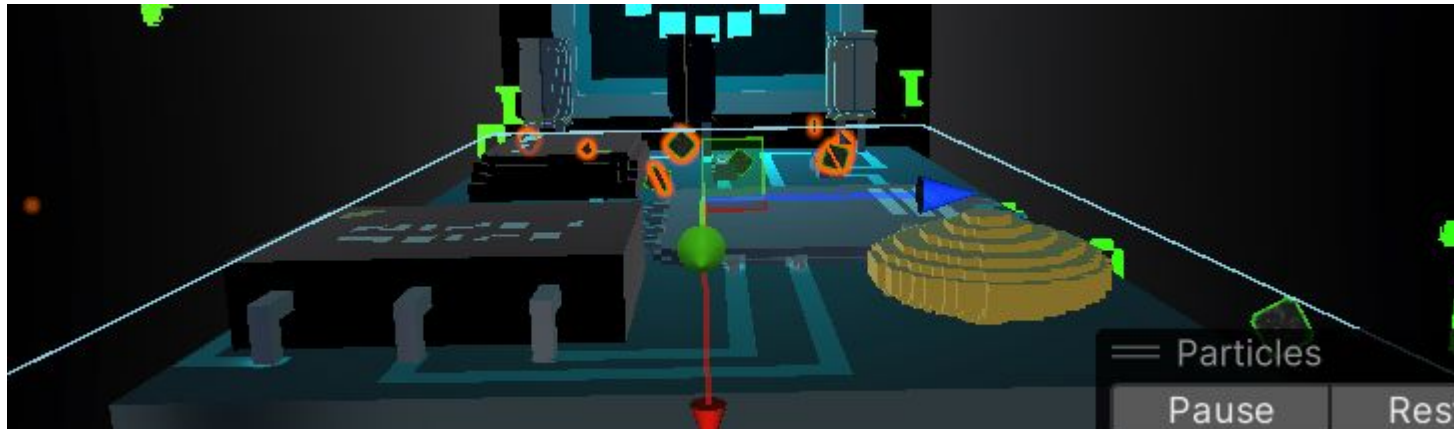
Again I Created a material and added the texture. After importing it here are the changes I made:

Things I enabled:

- Size over lifetime
 - Used a custom curve
- Rotation over lifetime
 - Custom curve again.

Things Changed:

- Initial Speed to random constants that are very low. (imitates dust)
- Initial rotation to be random (adds flavour)



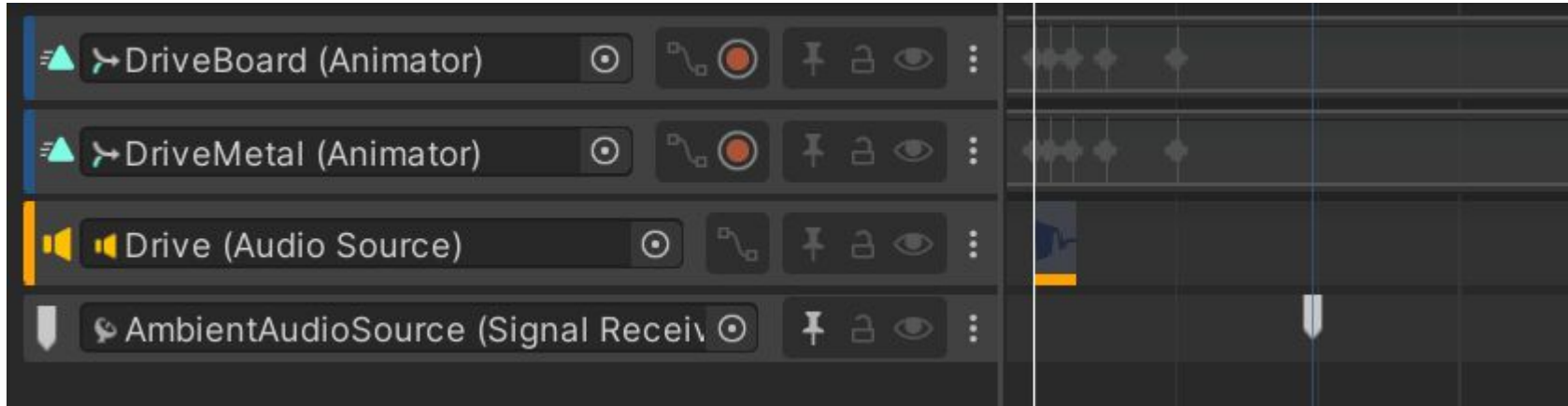
Animation - Door

Since I set the pivots of the two sides in modeling, animating them was quite easy. I simply added a timeline animated them opening, messed with the curves a little to make the animation more aggressive, as though it was getting hacked into.

I also added a little hit sound effect that would trigger when the doors would open.

I also used a signal to start the ambient audio once the player actually broke into the world, as it doesn't really make much sense otherwise.

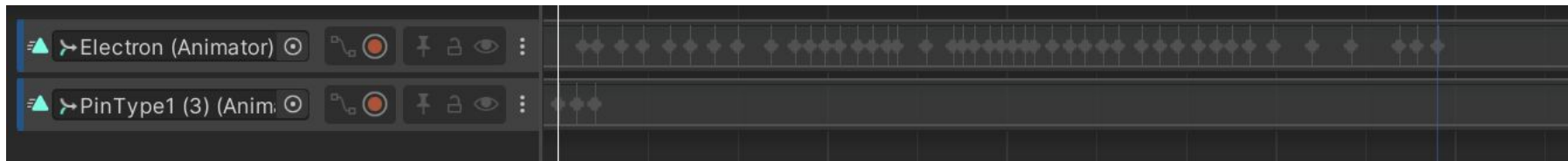
There's more information on sound later.



Animation - Electrons

Similar to the doors, I Grabbed a bunch of the aforementioned electrons and made them do parkour tricks around the map.

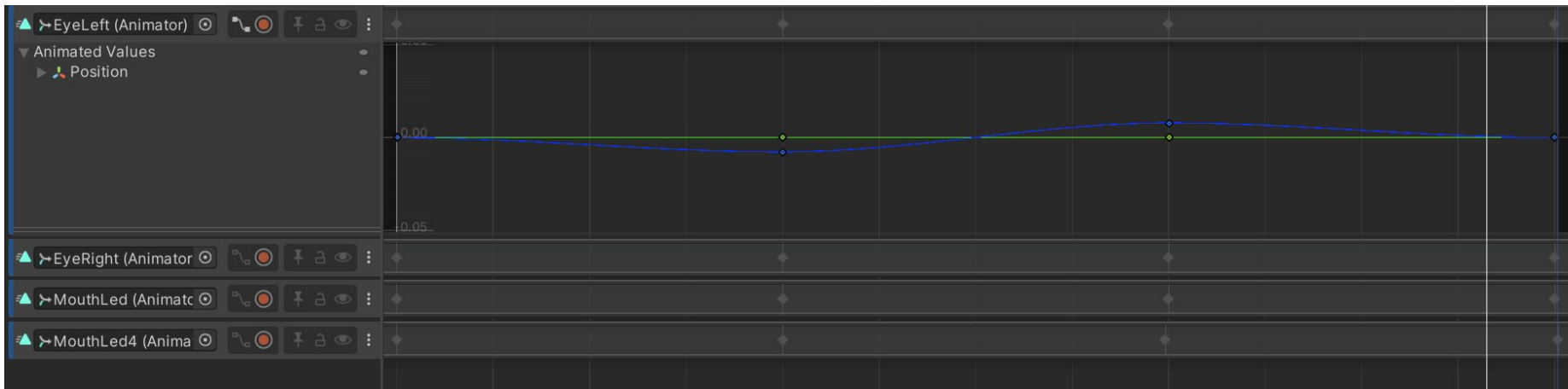
I kind of did this completely on the impression that the environment gave me and tried to make it a little more fun.



Animation - Face

Here again I experimented with the timeline and made what I would call an idle animation.

I think if I had some more time I would probably add a aggression animation that would trigger from the player getting too close to the board or screen.



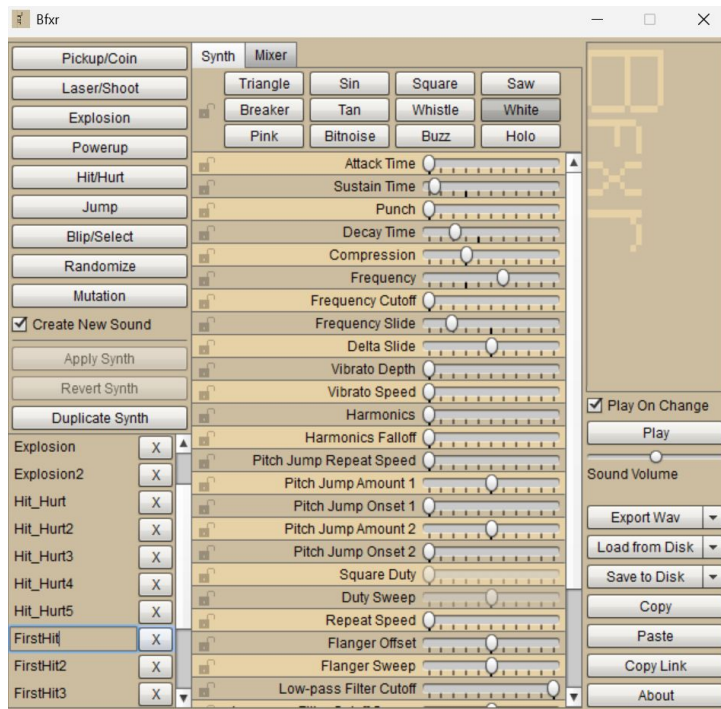
Sound

For the Hit SFX I used a Free Open Source Software called [BFXR](https://bfxr.sourceforge.io/) which generates 8-bit/retro sound effects with a randomiser. So I used it to generate my Hit SFX.

I got the Ambient sound from freesound.org with a creative Commons 0 license

Itinerantur by Vrymaa -- <https://freesound.org/s/785265/> --
License: Creative Commons 0

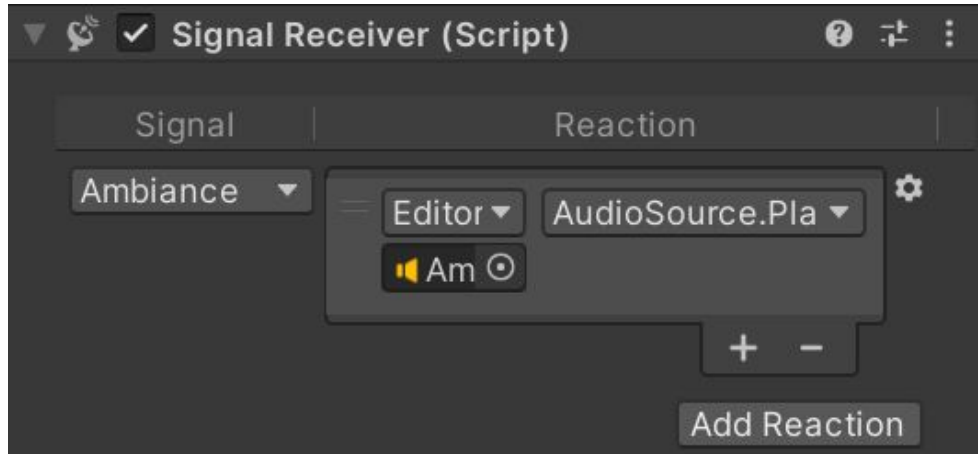
49588.wav by Vrezerino -- <https://freesound.org/s/42694/> --
-- License: Creative Commons 0



Sound - Signalling

As mentioned earlier I used a signal to trigger the ambient sound coming from the world.

This was quite simple to set up and I didnt even look at a tutorial or instructions. It just kind of worked very intuitively.

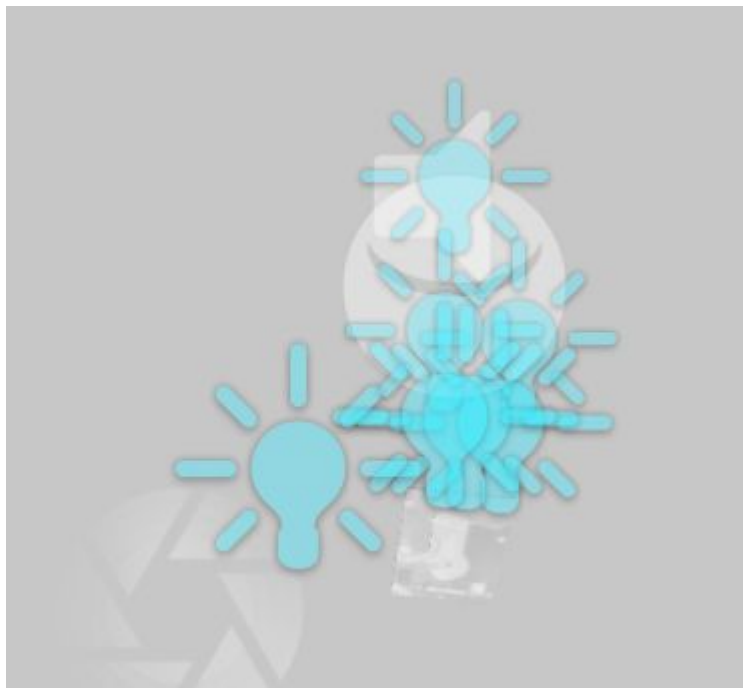


Lighting

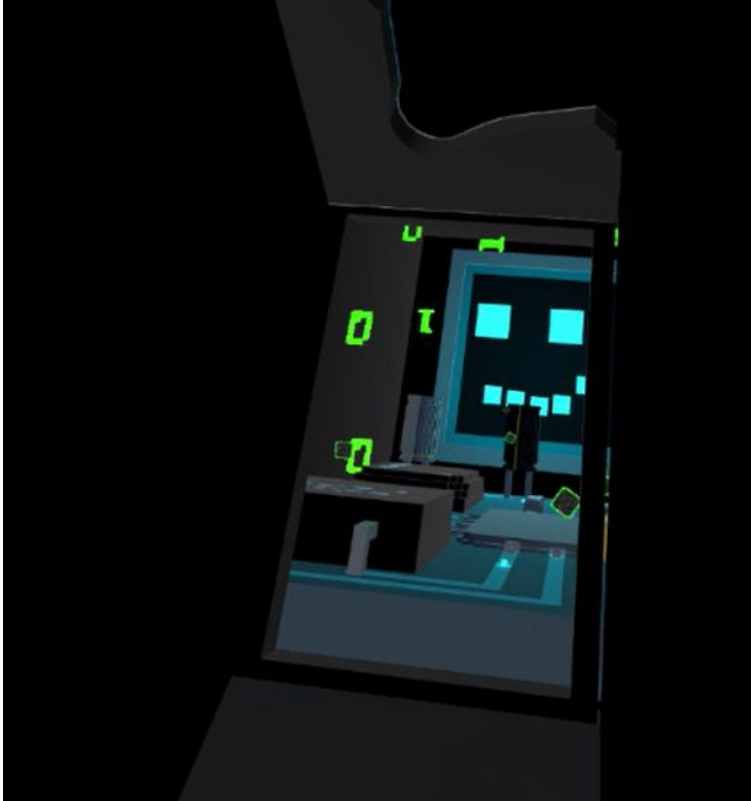
I kept Lighting simple and linked a bunch of little area lights to each electron, and one big area light to the main screen (imitating the halo from a screen)

I added some post processing and the reflection probe to calculate the lighting.

LIGHTS — — — — — →



Final Outcome.



<https://youtu.be/ddLiTS7DFNk>